# 5 · Exp 5 + 6 — Matrix multiplication and transpose (modular programming)

Two experiments, one pattern: three functions (input, operate, display) plus a main() that calls them. New C++ here: 2-D arrays and passing them to functions.

2-D array → Pass to a function → inputMatrix → multiply / transpose → displayMatrix

## 5.1 New C++: 2-D arrays

`int A[10][10];` is a grid of 10 rows × 10 columns; `A[i][j]` is row i, column j (both from 0). Like `arr[100]`, the size is the **maximum**: the user's 2 × 3 matrix uses only the top-left corner.

Walking a grid takes **two nested loops**: the outer loop picks the row, the inner loop walks along it.

```
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        cin >> A[i][j];
```

**WHY THE PARAMETER IS `INT MATRIX[][10]`**

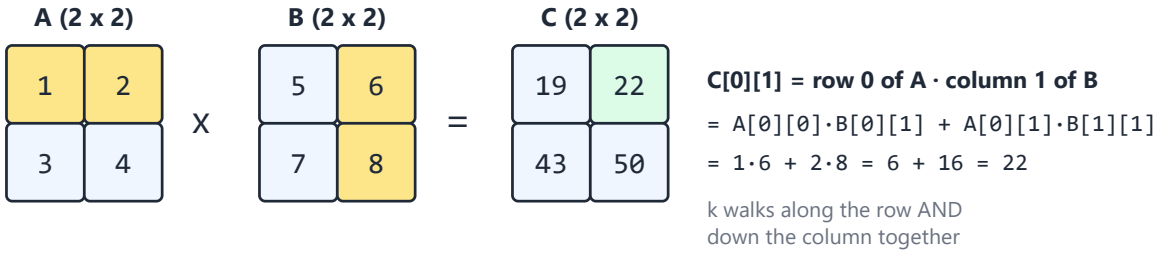
When passing a 2-D array, the **number of columns must be written** and must match the declaration (`[10]`). Only the first size may be left empty. `int matrix[][]` doesn't compile.

**Modular programming** means splitting the program into functions that each do one job: `inputMatrix()`, `multiplyMatrix()` / `transposeMatrix()`, `displayMatrix()`, with `main()` only reading sizes and calling them. The manual asks for exactly this structure, so use these exact function names.

## 5.2 Exp 5: multiplication

**GUESS FIRST**

A is 2 × **3** and B is **3** × 2. Can you multiply A × B? What order is the answer? Can you multiply B × A?




Rule: (m x n) times (n x p) gives (m x p). The two inner numbers must match: c1 == r2.

Each answer cell is one row of A "dotted" with one column of B.

$$C[i][j] = A[i][0]\cdot B[0][j] + A[i][1]\cdot B[1][j] + \ldots + A[i][c1-1]\cdot B[c1-1][j]$$

That sum is the **third loop** (k). Three nested loops: i over rows of A, j over columns of B, k along the shared dimension. Hence **$O(n^3)$** for n × n matrices.

E05_MATRIX_MULTIPLY.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to multiply two matrices: C = A x B
void multiplyMatrix(int A[][10], int B[][10], int C[][10], int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)              // each row of A
    {
        for (int j = 0; j < c2; j++)          // each column of B
        {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++)      // walk along row i and column j
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // FIX: arrays are 10 x 10, so larger sizes would overflow
    if (r1 < 1 || c1 < 1 || r2 < 1 || c2 < 1 || r1 > 10 || c1 > 10 || r2 > 10 || c2 > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    // Check multiplication condition
    if (c1 != r2)
```

```
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }

    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";
    displayMatrix(C, r1, c2);

    return 0;
}
```

The manual's sample (2 × 2), and a non-square case worked by hand (row 1 of the answer: 1·7 + 2·9 + 3·11 = 58):

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

Resultant Matrix:
19 22
43 50
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 3 2
Enter elements of first matrix:
1 2 3
4 5 6
Enter elements of second matrix:
7 8
9 10
11 12

Resultant Matrix:
58 64
139 154
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 2 3
Matrix multiplication not possible.
```

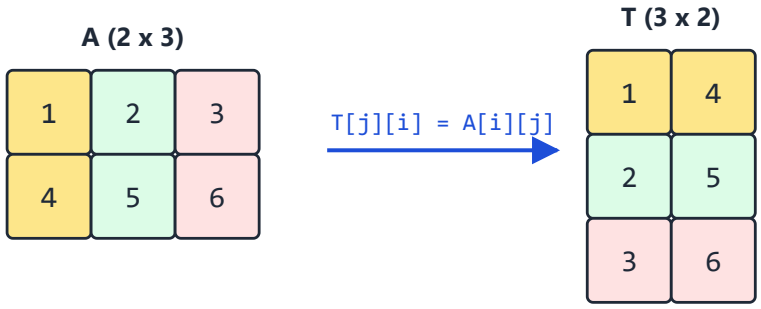
**THE LINE PEOPLE FORGET**

`C[i][j] = 0;` before the k-loop. Without it, C starts with garbage and the sum is wrong. It must sit **inside** the j-loop, so each cell starts from 0.

## 5.3 Exp 6: transpose

**GUESS FIRST**

A is 2 × 3. What is the order of its transpose, and where does A[0][2] go?




Column j of A becomes row j of T (each colour = one column of A → one row of T)

Rows become columns: T[j][i] = A[i][j].

E06_MATRIX_TRANSPOSE.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            T[j][i] = A[i][j];              // row i becomes column i
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}
```

```
int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    if (rows < 1 || cols < 1 || rows > 10 || cols > 10)   // FIX: array is 10 x 10
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);            // rows and columns swap

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3
4 5 6

Original Matrix:
1 2 3
4 5 6

Transpose Matrix:
1 4
2 5
3 6
```

Note the call `displayMatrix(T, cols, rows)`: the transpose has the sizes **swapped**. Passing `(T, rows, cols)` prints the wrong shape.

**WRITE IN YOUR RECORD — EXP 5**

**Aim:** To multiply two matrices using separate functions for input, multiplication and display.

**Algorithm:** read r1, c1, r2, c2 → if c1 ≠ r2, print "not possible" and stop → `inputMatrix(A)`, `inputMatrix(B)` → `multiplyMatrix`: for each i, j set C[i][j] = 0, then for k = 0…c1−1 add A[i][k]·B[k][j] → `displayMatrix(C, r1, c2)`.

**Complexity:** time O(r1·c1·c2), i.e. O($n^3$) for square matrices; space O(r1·c2) for C.

**Result:** [[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]].

**WRITE IN YOUR RECORD — EXP 6**

**Aim:** To find the transpose of a matrix using separate functions for input, transpose and display.

**Algorithm:** read rows, cols → `inputMatrix(A)` → `transposeMatrix`: for each i, j set T[j][i] = A[i][j] → display A (rows × cols) and T (cols × rows).

**Complexity:** time O(m·n); space O(m·n) for T.

**Result:** [[1,2,3],[4,5,6]] (2 × 3) → [[1,4],[2,5],[3,6]] (3 × 2).

## 5.4 Viva questions

| Question | Answer |
|---|---|
| Condition for A × B? | Columns of A = rows of B (c1 == r2). Result is r1 × c2. |
| Is A × B = B × A? | Generally no; matrix multiplication isn't commutative (B × A may not even be defined). |
| Why three loops? | Two pick the answer cell (i, j); the third (k) adds up the row·column products. |
| Why `int matrix[][10]`? | The compiler needs the column count to find `matrix[i][j]` in memory (row i starts i·10 elements in). |
| Advantage of modular programming? | Each function is small and testable; `inputMatrix` is reused for A and B; easier to debug and modify. |
| Can a square matrix be transposed in place? | Yes: swap A[i][j] with A[j][i] for j > i only (the upper triangle). A rectangular one can't, because its shape changes. |
| What is a symmetric matrix? | A square matrix equal to its transpose (A[i][j] = A[j][i]). |

## 5.5 If the examiner changes the question

"Multiply two 3 × 3 matrices", "check whether they can be multiplied" and "transpose of a 3 × 3" are all the programs above (type 3 3 as the sizes). The new ones:

**Add / subtract** (same order needed: r1 == r2 and c1 == c2, checked in main):

```
// Both matrices must have the SAME order (rows x cols)
void addMatrix(int A[][10], int B[][10], int S[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            S[i][j] = A[i][j] + B[i][j];
}
```

```
void subtractMatrix(int A[][10], int B[][10], int D[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            D[i][j] = A[i][j] - B[i][j];
}
```

**Symmetric or not:**

```
// Symmetric = square AND A[i][j] == A[j][i] everywhere (A equals its transpose)
bool isSymmetric(int A[][10], int rows, int cols)
{
    if (rows != cols)
        return false;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (A[i][j] != A[j][i])
                return false;
    return true;
}
```

In `main`: `if (isSymmetric(A, rows, cols)) cout << "Symmetric"; else cout << "Not symmetric";`

> **CHECK 5.1**
> A = [[2, 0], [1, 3]], B = [[1, 4], [5, 2]]. Compute C = A × B by hand, cell by cell.

> **CHECK 5.2**
> In `multiplyMatrix`, someone writes `for (int k = 0; k < c2; k++)`. For 2 × 3 times 3 × 2 matrices, what goes wrong?

## Answers

**Check 5.1:** C[0][0] = 2·1 + 0·5 = 2; C[0][1] = 2·4 + 0·2 = 8; C[1][0] = 1·1 + 3·5 = 16; C[1][1] = 1·4 + 3·2 = 10 → **[[2, 8], [16, 10]]**.

**Check 5.2:** k runs only to 1 (c2 = 2) instead of 2, so the third product A[i][2]·B[2][j] is never added and every cell is wrong. k must run over the **shared** size, c1 (= r2).